Concept traits should be named after concepts

Document #:	P1871R1
Date:	2019-11-07
Project:	Programming Language C++ LEWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

1 Revision History

R0 [P1871R0] was presented in Belfast and originally proposed making all of the variable template opt-ins for concepts named `enable_NAME`. LEWG rejected this direction. However, one of the other changes in the paper was to ensure at least that the traits were named the same as the concepts. This used to be the case, except when we renamed the concept to `sized_sentinel_for` we kept the trait named `disable_sized_sentinel`, so this paper proposes simply updating that trait to have the same name - so that all such traits are either `enable_NAME` or `disable_NAME`.

2 Motivation

The concept `sized_sentinel_for`, from [iterator.concept.sizedsentinel], is defined as:

```
template<class S, class I>
  concept sized_sentinel_for =
    sentinel_for<S, I> &&
    !disable_sized_sentinel<remove_cv_t<S>, remove_cv_t<I>> &&
    requires(const I& i, const S& s) {
      { s - i } -> same_as<iter_difference_t<I>>;
      { i - s } -> same_as<iter_difference_t<I>>;
    };
```

Note that the concept is named `sized_sentinel_for` but the trait is named `disable_sized_sentinel`. These used to be closely related because the concept used to be named `SizedSentinel` and got the suffix `_for` added as part of [P1754R1].

This paper proposes that the trait and the concept should have the same name.

3 Wording

Change 23.2 [iterator.synopsis]:

```
    // [iterator.concept.sizedsentinel], concept sized_sentinel_for
    template<class S, class I>
-     inline constexpr bool disable_sized_sentinel = false;
+     inline constexpr bool disable_sized_sentinel_for = false;
```

and later:

```
          template<class Iterator1, class Iterator2>
              requires (!sized_sentinel_for<Iterator1, Iterator2>)
        -   inline constexpr bool
disable_sized_sentinel<reverse_iterator<Iterator1>,
        -
reverse_iterator<Iterator2>> = true;
        +   inline constexpr bool
disable_sized_sentinel_for<reverse_iterator<Iterator1>,
        +
reverse_iterator<Iterator2>> = true;
```

Change 23.3.4.8 [iterator.concept.sizedsentinel]:

1 The `sized_sentinel_for` concept specifies requirements on an `input_or_output_iterator` and a corresponding `sentinel_for` that allow the use of the `-` operator to compute the distance between them in constant time.

```
        template<class S, class I>
          concept sized_sentinel_for =
            sentinel_for<S, I> &&
        -   !disable_sized_sentinel<remove_cv_t<S>, remove_cv_t<I>> &&
        +   !disable_sized_sentinel_for<remove_cv_t<S>, remove_cv_t<I>> &&
            requires(const I& i, const S& s) {
              { s - i } -> same_as<iter_difference_t<I>>;
              { i - s } -> same_as<iter_difference_t<I>>;
            };
```

2 Let `i` be an iterator of type `I`, and `s` a sentinel of type `S` such that `[i, s)` denotes a range. Let `N` be the smallest number of applications of `++i` necessary to make `bool(i == s)` be `true`. `S` and `I` model `sized_sentinel_for<S, I>` only if

```
        - [2.1]{.pnum} If `N` is representable by `iter_difference_t<I>`,
then `s - i` is well-defined and equals `N`.
        - [2.2]{.pnum} If `−N` is representable by `iter_difference_t<I>`,
then `i - s` is well-defined and equals `−N`.
```

```
          template<class S, class I>
        -   inline constexpr bool disable_sized_sentinel = false;
        +   inline constexpr bool disable_sized_sentinel_for = false;
```

3 *Remarks*: Pursuant to [namespace.std], users may specialize ~~`disable_sized_sentinel`~~ `disable_sized_sentinel_for` for *cv*-unqualified non-array object types `S` and `I` if `S` and/or `I` is a program-defined type. Such specializations shall be usable in constant expressions ([expr.const]) and have type `const bool`.

4 [*Note*: ~~`disable_sized_sentinel`~~ `disable_sized_sentinel_for` allows use of sentinels and iterators with the library that satisfy but do not in fact model `sized_sentinel_for`. — *end note*]

§ 4 References

[P1754R1] Herb Sutter, Casey Carter, Gabriel Dos Reis, Eric Niebler, Bjarne Stroustrup, Andrew Sutton, Ville Voutilainen. 2019. Rename concepts to standard_case for C++20, while we still can. https://wg21.link/p1754r1

[P1871R0] Barry Revzin. 2019. Should concepts be enabled or disabled? https://wg21.link/p1871r0